Ejercicios Evaluables — 02MIAR: Matemáticas para la IA

Máster Universitario en Inteligencia Artificial — Universidad Internacional de Valencia

Todo implementado desde cero en Python puro. `numpy` solo en el apartado **1g** donde el enunciado lo exige explícitamente.

Ejercicio	Tema	Puntos
1	Cálculo del determinante	5 pts
2	Descenso de gradiente	5 pts

In [48]:
```
import sys
import math
import time
import numpy as np

# Forzar UTF-8 en consola Windows (evita UnicodeEncodeError)
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8', errors='replace')
```

Ejercicio 1: Cálculo del Determinante

El objetivo es ilustrar el impacto computacional de elegir distintos algoritmos para calcular $\det(A)$: expansión de Laplace recursiva $\mathcal{O}(n!)$ vs eliminación de Gauss $\mathcal{O}(n^3)$.

1a) `determinante_recursivo` — Expansión de Laplace [1 punto]

> **Enunciado:** Implementa `determinante_recursivo` usando la definición recursiva de Laplace.

Expansión por la primera fila:

$$\det(A) = \sum_{j=0}^{n-1} (-1)^j \cdot a_{0j} \cdot \det(M_{0j})$$

donde M_{0j} es la submatriz $(n-1) \times (n-1)$ al eliminar fila 0 y columna j.

Casos base: $n = 1 \Rightarrow \det = a_{00}$; $n = 2 \Rightarrow \det = ad - bc$.

Complejidad: $T(n) = n \cdot T(n-1) \Rightarrow \mathcal{O}(n!)$.

In [49]:
```python
def _copiar(A):
    # Copia profunda de lista de listas
    return [fila[:] for fila in A]

def _menor(A, fila, col):
    # Submatriz (n-1)x(n-1) eliminando 'fila' y 'col'
    return [
        [A[i][j] for j in range(len(A[i])) if j != col]
        for i in range(len(A)) if i != fila
    ]

def determinante_recursivo(A):
    # det(A) por expansion de Laplace en la primera fila. O(n!)
    n = len(A)
    if n == 1:
        return A[0][0]
    if n == 2:
        return A[0][0]*A[1][1] - A[0][1]*A[1][0]
    det = 0.0
    for j in range(n):
        signo = 1 if j % 2 == 0 else -1
        det += signo * A[0][j] * determinante_recursivo(_menor(A, 0, j))
    return det

# --- Verificacion ---
A_test = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 10]]

print(f"det(A_test) = {determinante_recursivo(A_test)}")   # Esperado: -3.0
print(f"Verificacion numpy: {np.linalg.det(A_test):.6f}")
```

```
det(A_test) = -3.0
Verificacion numpy: -3.000000
```

1b) Determinante de Matriz Triangular [0.5 puntos]

> **Enunciado:** Si A es triangular (superior o inferior), ¿existe forma directa de calcular su determinante? Justifíquese.

Sí. El determinante es el **producto de los elementos de la diagonal principal**:

$$\boxed{\det(A) = \prod_{i=0}^{n-1} a_{ii}}$$

Justificación: En una matriz triangular superior, al expandir por la primera columna el único elemento no nulo es a_{00}, de modo que $\det(A) = a_{00} \cdot \det(A_{1:,1:})$. Aplicando el argumento recursivamente se obtiene el producto de la diagonal. Para la triangular inferior el razonamiento es idéntico expandiendo por la primera fila.

Complejidad: $\mathcal{O}(n)$.

In [50]:
```python
def determinante_triangular(A):
    # det(A) para A triangular: producto de la diagonal. O(n)
    det = 1.0
    for i in range(len(A)):
```

```
        det *= A[i][i]
    return det

# --- Ejemplo: matriz triangular superior 3x3 ---
T = [[2, 3, 5],
     [0, 4, 1],
     [0, 0, 7]]

print(f"det(T) triangular   = {determinante_triangular(T)}")       # 2*4*7 = 56
print(f"det(T) Laplace      = {determinante_recursivo(T)}")        # debe coinci
print(f"det(T) numpy        = {np.linalg.det(T):.6f}")
```

```
det(T) triangular   = 56.0
det(T) Laplace      = 56.0
det(T) numpy        = 56.000000
```

1c) Operaciones Elementales sobre el Determinante [0.5 puntos]

> **Enunciado:** ¿Cómo alteran el determinante las dos operaciones elementales siguientes: (1) intercambiar dos filas, (2) sumar a una fila otra fila multiplicada por un escalar α?

Operación 1 — Intercambiar dos filas:

$$\boxed{\det(A') = -\det(A)}$$

Al intercambiar dos filas se compone cada permutación σ con una transposición, invirtiendo su signo en la suma de Leibniz. El determinante cambia de signo.

Operación 2 — Sumar $\alpha\times$ fila$_k$ a fila$_i$:

$$\boxed{\det(A') = \det(A)}$$

Por multilinealidad, la fila resultante se descompone en la fila original (contribuye $\det(A)$) más α veces una fila duplicada (contribuye 0, pues un determinante con dos filas iguales es nulo). Por tanto $\det(A') = \det(A) + 0 = \det(A)$.

In [51]:
```
# Demostracion numerica — apartado 1c

B = [[3.0, 1.0, 2.0],
     [0.0, 4.0, 5.0],
     [1.0, 2.0, 6.0]]

det_B = determinante_recursivo(B)
print(f"det(B) original = {det_B:.6f}")

# ── Operacion 1: swap fila 0 <-> fila 2 ──────────────────────────────
B_swap = _copiar(B)
B_swap[0], B_swap[2] = B_swap[2][:], B_swap[0][:]
det_swap = determinante_recursivo(B_swap)
print(f"\nOperacion 1 — swap fila 0 <-> fila 2:")
print(f"  det(B_swap)   = {det_swap:.6f}")
print(f"  -det(B)       = {-det_B:.6f}")
print(f"  Signo invertido: {abs(det_swap - (-det_B)) < 1e-10}")
```

```
# ── Operacion 2: fila 1 += alpha * fila 0 (alpha arbitrario) ──────────
alpha = 5.0
B_suma = _copiar(B)
for j in range(len(B[0])):
    B_suma[1][j] += alpha * B[0][j]
det_suma = determinante_recursivo(B_suma)
print(f"\nOperacion 2 — fila 1 += {alpha:.0f} * fila 0:")
print(f"  det(B_suma)   = {det_suma:.6f}")
print(f"  det(B)        = {det_B:.6f}")
print(f"  Det invariante: {abs(det_suma - det_B) < 1e-10}")
```

```
det(B) original = 39.000000

Operacion 1 — swap fila 0 <-> fila 2:
  det(B_swap)   = -39.000000
  -det(B)       = -39.000000
  Signo invertido: True

Operacion 2 — fila 1 += 5 * fila 0:
  det(B_suma)   = 39.000000
  det(B)        = 39.000000
  Det invariante: True
```

1d) Eliminación de Gauss con Pivoteo Parcial [1 punto]

> **Enunciado:** Investiga e implementa la eliminación de Gauss con pivoteo parcial para convertir A en triangular inferior.

Algoritmo — se procesan columnas de **derecha a izquierda** ($k = n-1, \dots, 1$):

1. **Pivoteo parcial:** buscar la fila $r \in \{0, \dots, k\}$ con mayor $|a_{rk}|$ y hacer swap $k \leftrightarrow r$ (mejora estabilidad numérica y evita divisiones por cero).
2. **Eliminación:** para cada fila $i < k$, anular a_{ik}:

$$a_{ij} \leftarrow a_{ij} - \frac{a_{ik}}{a_{kk}} \cdot a_{kj}, \quad j = 0, \dots, n-1$$

Al finalizar, L es triangular inferior: todos los elementos sobre la diagonal son cero.
Complejidad: $\mathcal{O}(n^3)$.

In [52]:
```
def escalonamiento_gauss(A):
    """Eliminacion de Gauss con pivoteo parcial -> triangular INFERIOR L.
    Procesa columnas de derecha a izquierda eliminando entradas sobre la diagona
    Devuelve (L, num_swaps).
    """
    n = len(A)
    L = _copiar(A)
    num_swaps = 0

    for k in range(n - 1, 0, -1):

        # Pivoteo parcial: fila r en 0..k con mayor |L[r][k]|
        max_abs, max_fila = abs(L[k][k]), k
        for i in range(k):
            if abs(L[i][k]) > max_abs:
                max_abs, max_fila = abs(L[i][k]), i
        if max_fila != k:
            L[k], L[max_fila] = L[max_fila], L[k]
```

```
            num_swaps += 1

        if abs(L[k][k]) < 1e-14:
            continue

        # Eliminacion: anular entradas sobre la diagonal en columna k
        for i in range(k):
            if abs(L[i][k]) < 1e-15:
                continue
            m = L[i][k] / L[k][k]
            for j in range(n):
                L[i][j] -= m * L[k][j]
            L[i][k] = 0.0

    return L, num_swaps

# --- Demostracion ---
A_demo = [[2.0, 1.0, -1.0],
          [-3.0, -1.0, 2.0],
          [-2.0, 1.0, 2.0]]

Lmat, swaps = escalonamiento_gauss(A_demo)
print("Matriz original A:")
for fila in A_demo:
    print(" ", fila)
print(f"\nForma triangular INFERIOR L ({swaps} swap(s)):")
for fila in Lmat:
    print(" ", [round(x, 6) for x in fila])

n = len(Lmat)
es_tri_inf = all(abs(Lmat[i][j]) < 1e-10 for i in range(n) for j in range(i+1, n
print(f"\n¿Es triangular inferior? {es_tri_inf}")
```

```
Matriz original A:
  [2.0, 1.0, -1.0]
  [-3.0, -1.0, 2.0]
  [-2.0, 1.0, 2.0]

Forma triangular INFERIOR L (0 swap(s)):
  [0.25, 0.0, 0.0]
  [-1.0, -2.0, 0.0]
  [-2.0, 1.0, 2.0]

¿Es triangular inferior? True
```

1e) `determinante_gauss` [0.5 puntos]

> **Enunciado:** ¿Cómo calcular el determinante usando la eliminación gaussiana? Implementa `determinante_gauss`.

A partir de los resultados de los apartados anteriores:

- Cada swap invierte el signo del determinante (1c, Op. 1).
- La eliminación por filas no lo altera (1c, Op. 2).
- $\det(L) = \prod_i L_{ii}$ (1b).

$$\boxed{\det(A) = (-1)^{\text{num_swaps}} \cdot \prod_{i=0}^{n-1} L_{ii}}$$

Complejidad: $\mathcal{O}(n^3)$.

In [53]:
```python
def determinante_gauss(A):
    # det(A) mediante eliminacion gaussiana con pivoteo parcial. Complejidad: O(
    L, num_swaps = escalonamiento_gauss(A)
    det = 1.0
    for i in range(len(L)):
        det *= L[i][i]                  # producto de la diagonal de L
    if num_swaps % 2 == 1:
        det = -det                      # corregir signo segun numero de interca
    return det

# --- Verificacion: Laplace = Gauss = numpy ---
matrices_test = [
    ([[2, 1], [3, 4]],                  "2x2"),
    ([[1, 2, 3], [4, 5, 6], [7, 8, 10]], "3x3"),
    (A_demo,                             "A_demo"),
]
print(f"{'Matriz':>8}  {'Laplace':>14}  {'Gauss':>14}  {'numpy':>14}  {'OK?':>6}
print("-" * 62)
for M, nombre in matrices_test:
    dr = determinante_recursivo(M)
    dg = determinante_gauss(M)
    dn = float(np.linalg.det(M))
    ok = abs(dr - dg) < 1e-9 and abs(dr - dn) < 1e-9
    print(f"{nombre:>8}  {dr:>14.8f}  {dg:>14.8f}  {dn:>14.8f}  {str(ok):>6}")
```

```
 Matriz         Laplace           Gauss           numpy     OK?
--------------------------------------------------------------
    2x2      5.00000000      5.00000000      5.00000000    True
    3x3     -3.00000000     -3.00000000     -3.00000000    True
 A_demo     -1.00000000     -1.00000000     -1.00000000    True
```

1f) Complejidad Computacional [0.5 puntos]

> **Enunciado:** Obtén la complejidad del cálculo del determinante con la definición recursiva de Laplace y con la eliminación de Gauss con pivoteo parcial.

Método	Recurrencia	Complejidad
Laplace	$T(n) = n \cdot T(n-1)$	$\mathcal{O}(n!)$
Gauss	$\approx n^3/3$ operaciones	$\mathcal{O}(n^3)$

Laplace: cada nivel genera n subproblemas de tamaño $n-1$, lo que produce $T(n) = n!$.

Gauss: el doble bucle de eliminación realiza $\sum_{k=1}^{n-1} k \approx n^2/2$ filas, cada una con $\mathcal{O}(n)$ trabajo, dando $\mathcal{O}(n^3)$.

Para $n = 20$: Laplace requiere $\sim 2.4 \times 10^{18}$ operaciones; Gauss, $\sim 2{,}667$.

1g) Tabla Comparativa de Tiempos [1 punto]

> **Enunciado:** Genera matrices $A_n \in \mathbb{R}^{n \times n}$ para $2 \leq n \leq 10$ con `numpy.random.rand` y compara los tiempos de `determinante_recursivo`, `determinante_gauss` y `numpy.linalg.det`.

Semilla fija `seed = 42 + n` para reproducibilidad. Tiempos medidos con `time.perf_counter`.

In [54]:
```
def _np_a_lista(A_np):
    return [list(row) for row in A_np]

print(f"{'n':>3}  {'det_recursivo':>15}  {'t_rec(s)':>11}  "
      f"{'det_gauss':>15}  {'t_gauss(s)':>11}  "
      f"{'det_numpy':>15}  {'t_numpy(s)':>11}")
print("-" * 90)

for n in range(2, 11):
    np.random.seed(42 + n)
    A_np = np.random.rand(n, n)
    A    = _np_a_lista(A_np)

    t0 = time.perf_counter(); dr = determinante_recursivo(A); t_rec   = time.per
    t0 = time.perf_counter(); dg = determinante_gauss(A);     t_gauss = time.per
    t0 = time.perf_counter(); dn = float(np.linalg.det(A_np));t_np    = time.per

    print(f"{n:>3}  {dr:>15.8f}  {t_rec:>11.6f}  "
          f"{dg:>15.8f}  {t_gauss:>11.6f}  "
          f"{dn:>15.8f}  {t_np:>11.6f}")

print("-" * 90)
```

```
  n    det_recursivo     t_rec(s)        det_gauss   t_gauss(s)        det_numpy
t_numpy(s)
-------------------------------------------------------------------------------
---------
  2       0.22292587     0.000009       0.22292587     0.000020       0.22292587
0.000091
  3      -0.02023684     0.000014      -0.02023684     0.000011      -0.02023684
0.000021
  4       0.28578379     0.000032       0.28578379     0.000013       0.28578379
0.000014
  5       0.31281626     0.000145       0.31281626     0.000024       0.31281626
0.000019
  6      -0.00001029     0.000813      -0.00001029     0.000037      -0.00001029
0.000055
  7       0.05025437     0.006980       0.05025437     0.000058       0.05025437
0.000067
  8      -0.01945368     0.065733      -0.01945368     0.000111      -0.01945368
0.000159
  9       0.00712359     0.726916       0.00712359     0.000164       0.00712359
0.000172
 10       0.00272335     7.433748       0.00272335     0.000185       0.00272335
0.000183
-------------------------------------------------------------------------------
---------
```

Interpretación:

- Para $n \leq 5$: los tres métodos son prácticamente instantáneos.
- Para $n \geq 7$: `determinante_recursivo` crece $\mathcal{O}(n!)$ — el tiempo se multiplica por $\sim n$ en cada paso ($n = 9$ tarda ~ 0.6 s, $n = 10$ tarda ~ 7.1 s frente a microsegundos de Gauss).
- `determinante_gauss` : tiempos estables en todo el rango $\mathcal{O}(n^3)$.
- `numpy.linalg.det` : el más rápido — implementación LAPACK en C, mismo $\mathcal{O}(n^3)$.
- Los tres métodos producen el mismo determinante (diferencias $< 10^{-14}$ por redondeo flotante).

Ejercicio 2: Descenso de Gradiente

2a) `descenso_gradiente` — Implementación general [1 punto]

> **Enunciado:** Implementa el descenso de gradiente para funciones de n variables con parámetros `grad_f` , `x0` , `γ` , `tol` y `maxit` .

Update rule:

$$x_{k+1} = x_k - \gamma \cdot \nabla f(x_k)$$

El proceso termina cuando $\|\nabla f(x_k)\|_2 < \text{tol}$ o se alcanza `maxit` .

2b) $f(x) = 3x^4 + 4x^3 - 12x^2 + 7$ [2 puntos]

Puntos críticos: $f'(x) = 12x(x+2)(x-1) = 0 \Rightarrow x \in \{-2,\ 0,\ 1\}$

Clasificación con $f''(x) = 36x^2 + 24x - 24$:

x	$f(x)$	$f''(x)$	Tipo
-2	-25	$72 > 0$	**Mínimo global**
0	7	$-24 < 0$	**Máximo local**
1	2	$36 > 0$	**Mínimo local**

In [55]:
```python
def norma_l2(v):
    # Norma euclidiana ||v||_2 = sqrt(sum vi^2)
    return sum(vi * vi for vi in v) ** 0.5

def descenso_gradiente(grad_f, x0, gamma, tol, maxit):
    # Descenso de gradiente para minimizar f: R^n -> R.
    # Devuelve (x, iteraciones, ||nabla_f(x)||_2)
    x, maxit = x0[:], int(maxit)
    n = len(x)

    for k in range(maxit):

        # Calcular gradiente; capturar overflow si los valores explotan
        try:
            g = grad_f(x)
        except OverflowError:
            return x, k, float('inf')

        norma = norma_l2(g)

        if not math.isfinite(norma):       # NaN o Inf en el gradiente = diverge
            return x, k, float('inf')

        if norma < tol:                    # criterio de convergencia
            return x, k, norma

        # Actualizacion: x_{k+1} = x_k - gamma * nabla_f(x_k)
        for i in range(n):
            x[i] -= gamma * g[i]

        if not all(math.isfinite(xi) for xi in x):   # x se fue a inf/nan
            return x, k + 1, float('inf')

    # Se agotaron las iteraciones
    try:
        norma = norma_l2(grad_f(x))
    except OverflowError:
        norma = float('inf')
    return x, maxit, norma

print("Funciones norma_l2 y descenso_gradiente definidas correctamente.")
```

```
Funciones norma_l2 y descenso_gradiente definidas correctamente.
```

Experimentos 2b-i a 2b-v

In [56]:
```
def f(x_vec):
    # f(x) = 3x^4 + 4x^3 - 12x^2 + 7
    x = x_vec[0]
    return 3*x**4 + 4*x**3 - 12*x**2 + 7

def grad_f(x_vec):
    # Derivada: f'(x) = 12x^3 + 12x^2 - 24x
    x = x_vec[0]
    return [12*x**3 + 12*x**2 - 24*x]

# Verificacion analitica de los puntos criticos
puntos = [(-2, 'Minimo global'), (0, 'Maximo local'), (1, 'Minimo local')]
print("{:>5}  {:>8}  {:>9}  {:>10}  {}".format('x', 'f(x)', "f'(x)", "f''(x)", '
print("-" * 55)
for xc, tipo in puntos:
    fx   = f([xc])
    dfx  = grad_f([xc])[0]
    ddfx = 36*xc**2 + 24*xc - 24
    print("{:>5}  {:>8.2f}  {:>9.4f}  {:>10.2f}  {}".format(xc, fx, dfx, ddfx, t
```

```
   x      f(x)      f'(x)      f''(x)  Tipo
-------------------------------------------------------
  -2    -25.00     0.0000       72.00  Minimo global
   0      7.00     0.0000      -24.00  Maximo local
   1      2.00     0.0000       36.00  Minimo local
```

2b-i) $x_0 = 3, \gamma = 0.001$ [0.5 puntos]

> **Enunciado:** Aplica el método sobre $f(x)$ con $x_0 = 3, \gamma = 0.001$, `tol=1e-12`, `maxit=1e5`.

$x_1 = 3 - 0.001 \cdot f'(3) = 3 - 0.001 \cdot 360 = 2.64 > 0$ → permanece en la cuenca $x > 0$ → el algoritmo converge al **mínimo local** $x = 1$, $f(1) = 2$.

In [57]:
```
x_i, k_i, ng_i = descenso_gradiente(grad_f, [3.0], gamma=0.001, tol=1e-12, maxit

print(f"x          = {x_i[0]:.10f}")
print(f"f(x)       = {f(x_i):.8f}")
print(f"Iteraciones: {k_i}    ||nabla_f|| = {ng_i:.2e}")
```

```
x          = 1.0000000000
f(x)       = 2.00000000
Iteraciones: 831    ||nabla_f|| = 9.91e-13
```

Interpretación: $x_1 = 2.64 > 0$ — el algoritmo permanece en la cuenca $x > 0$ y converge al **mínimo local** $x = 1$, $f(1) = 2$.

2b-ii) $x_0 = 3, \gamma = 0.01$ [0.5 puntos]

> **Enunciado:** Aplica el método sobre $f(x)$ con $x_0 = 3, \gamma = 0.01$, `tol=1e-12`, `maxit=1e5`.

$x_1 = 3 - 0.01 \cdot 360 = -0.6 < 0$ → cruza el máximo en $x = 0$ y cae en la cuenca $x < 0$ → converge al **mínimo global** $x = -2$, $f(-2) = -25$.

In [58]:
```
x_ii, k_ii, ng_ii = descenso_gradiente(grad_f, [3.0], gamma=0.01, tol=1e-12, max

print(f"x          = {x_ii[0]:.10f}")
print(f"f(x)       = {f(x_ii):.8f}")
print(f"Iteraciones: {k_ii}    ||nabla_f|| = {ng_ii:.2e}")
```

```
x          = -2.0000000000
f(x)       = -25.00000000
Iteraciones: 31    ||nabla_f|| = 8.38e-13
```

Interpretación: $x_1 = -0.6 < 0$ — el primer paso cruza el máximo en $x = 0$ y cae en la cuenca $x < 0$. El algoritmo converge al **mínimo global** $x = -2$, $f(-2) = -25$ en 31 iteraciones.

2b-iii) Contraste e interpretación [0.5 puntos]

> **Enunciado:** Contrasta e interpreta los resultados anteriores. ¿Qué influencia tiene la elección de γ?

γ	x_1	Cuenca	Resultado
0.001	$2.64 > 0$	$x > 0$	Mínimo local $x = 1$, $f = 2$
0.01	$-0.6 < 0$	$x < 0$	Mínimo global $x = -2$, $f = -25$

El learning rate γ no solo controla la velocidad de convergencia: un valor mayor puede cambiar la cuenca de atracción y, con ello, el mínimo al que converge el algoritmo. No existe un γ universalmente óptimo.

2b-iv) $x_0 = 3$, $\gamma = 0.1$ [0.5 puntos]

> **Enunciado:** Aplica el método sobre $f(x)$ con $x_0 = 3$, $\gamma = 0.1$. Interpreta el resultado.

$x_1 = 3 - 0.1 \cdot 360 = -33$ → $f'(-33) \approx -1.3 \times 10^5$ → $x_2 \approx +12{,}967$ → el proceso diverge.

$\gamma = 0.1$ viola la condición de convergencia $\gamma < 2/L$. El algoritmo oscila con amplitud creciente: **gradient explosion**.

In [59]:
```
x_iv, k_iv, ng_iv = descenso_gradiente(grad_f, [3.0], gamma=0.1, tol=1e-12, maxi

print(f"x          = {x_iv[0]:.4e}")
print(f"Iteraciones: {k_iv}    ||nabla_f|| = {ng_iv:.2e}")
```

```
x          = -5.9517e+125
Iteraciones: 5    ||nabla_f|| = inf
```

Interpretación: El algoritmo diverge en 5 iteraciones — **gradient explosion**. x escala hasta $\sim 10^{125}$ porque $\gamma = 0.1$ viola la condición de convergencia $\gamma < 2/L$.

2b-v) $x_0 = 0, \gamma = 0.001$ [0.5 puntos]

> **Enunciado:** Aplica el método sobre $f(x)$ con $x_0 = 0$. ¿Es deseable el resultado? ¿A qué se debe este fenómeno?

$f'(0) = 0 \Rightarrow \|\nabla f(0)\| = 0 < \text{tol}$ → el algoritmo para en 0 iteraciones sin moverse.

No es deseable. $x = 0$ es un máximo local ($f''(0) = -24 < 0$). El descenso de gradiente no distingue entre mínimos, máximos y puntos de silla: únicamente detecta que el gradiente es cero.

In [60]:
```
x_v, k_v, ng_v = descenso_gradiente(grad_f, [0.0], gamma=0.001, tol=1e-12, maxit

print(f"x          = {x_v[0]:.10f}")
print(f"f(x)       = {f(x_v):.8f}")
print(f"Iteraciones: {k_v}    ||nabla_f|| = {ng_v:.2e}")
```

```
x          = 0.0000000000
f(x)       = 7.00000000
Iteraciones: 0    ||nabla_f|| = 0.00e+00
```

Interpretación: El algoritmo para en 0 iteraciones porque $f'(0) = 0$ exactamente. **No es deseable**: $x = 0$ es un máximo local ($f''(0) = -24 < 0$). El descenso de gradiente no distingue mínimos de máximos ni puntos de silla — solo detecta $\nabla f = 0$.

2c) $g(x, y) = x^2 + y^3 + 3xy + 1$ [1.5 puntos]

Puntos críticos: $\nabla g = \mathbf{0}$

$$\frac{\partial g}{\partial x} = 2x + 3y = 0 \quad \frac{\partial g}{\partial y} = 3y^2 + 3x = 0$$

Sustituyendo $y = -2x/3$ en $x = -y^2$: $x = 0$ ó $x = -9/4$.

Hessiana: $H = \begin{pmatrix} 2 & 3 \\ 3 & 6y \end{pmatrix}$, $\det(H) = 12y - 9$

Punto	$\det(H)$	Tipo
$P_1 = (0,\ 0)$	$-9 < 0$	**Punto de silla**
$P_2 = (-9/4,\ 3/2)$	$9 > 0$	**Mínimo local**

In [61]:
```
def grad_g(xy):
    # nabla g(x,y) = (2x + 3y,  3y^2 + 3x)
```

```
    x, y = xy[0], xy[1]
    return [2*x + 3*y,  3*y**2 + 3*x]

# Verificacion analitica de los puntos criticos
P1 = [0.0, 0.0]
P2 = [-9/4, 3/2]
print(f"nabla_g en P1=(0,0):        {grad_g(P1)}   (esperado: [0, 0])")
print(f"nabla_g en P2=(-2.25,1.5):  {grad_g(P2)}   (esperado: [0.0, 0.0])")
```

```
nabla_g en P1=(0,0):        [0.0, 0.0]   (esperado: [0, 0])
nabla_g en P2=(-2.25,1.5):  [0.0, 0.0]   (esperado: [0.0, 0.0])
```

2c-i) $\mathbf{x}_0 = (-1, 1)$, $\gamma = 0.01$ [0.5 puntos]

> **Enunciado:** Aplica el método sobre $g(x, y)$ con $\mathbf{x}_0 = (-1, 1)$, $\gamma = 0.01$, `tol=1e-12`, `maxit=1e5`.

$\nabla g(-1, 1) = [1,\ 0] \neq \mathbf{0}$ → el algoritmo se desplaza hacia P_2 y converge al **mínimo local** $P_2 = (-9/4,\ 3/2)$. Resultado deseable.

In [62]:
```
xg1, kg1, ng1 = descenso_gradiente(grad_g, [-1.0, 1.0], gamma=0.01, tol=1e-12, m

print(f"(x, y)     = ({xg1[0]:.10f},  {xg1[1]:.10f})")
print(f"Esperado:    (-2.2500000000,   1.5000000000)")
print(f"Iteraciones: {kg1}    ||nabla_g|| = {ng1:.2e}")
```

```
(x, y)     = (-2.2500000000,  1.5000000000)
Esperado:    (-2.2500000000,   1.5000000000)
Iteraciones: 3139    ||nabla_g|| = 9.99e-13
```

Interpretación: El algoritmo converge al **mínimo local** $P_2 = (-9/4,\ 3/2)$ en 3139 iteraciones. Resultado correcto y deseable.

2c-ii) $\mathbf{x}_0 = (0, 0)$, $\gamma = 0.01$ [0.5 puntos]

> **Enunciado:** ¿Qué ocurre si $\mathbf{x}_0 = (0, 0)$? ¿Se obtiene un resultado deseable?

$\nabla g(0, 0) = [0,\ 0]$ → el algoritmo para en 0 iteraciones sin moverse.

No es deseable. $(0, 0)$ es un punto de silla ($\det(H) = -9 < 0$), no un mínimo.

In [63]:
```
xg2, kg2, ng2 = descenso_gradiente(grad_g, [0.0, 0.0], gamma=0.01, tol=1e-12, ma

print(f"(x, y)     = ({xg2[0]:.10f},  {xg2[1]:.10f})")
print(f"Iteraciones: {kg2}    ||nabla_g|| = {ng2:.2e}")
```

```
(x, y)     = (0.0000000000,  0.0000000000)
Iteraciones: 0    ||nabla_g|| = 0.00e+00
```

Interpretación: El algoritmo para en 0 iteraciones porque $\nabla g(0, 0) = [0,\ 0]$. **No es deseable**: $(0, 0)$ es un punto de silla ($\det(H) = -9 < 0$), no un mínimo.

2c-iii) Estudio analítico y contraste [0.5 puntos]

> **Enunciado:** Realiza el estudio analítico de g y utilízalo para explicar y contrastar los resultados anteriores.

Punto	∇g	$\det(H)$	Clasificación
$P_1 = (0, 0)$	$(0, 0)$	$-9 < 0$	**Punto de silla**
$P_2 = (-9/4, 3/2)$	$(0, 0)$	$9 > 0$	**Mínimo local**

- **2c-i** ($x_0 = (-1, 1)$): $\nabla g \neq 0$ → el algoritmo se mueve → converge a P_2. Correcto.
- **2c-ii** ($x_0 = (0, 0)$): $\nabla g = 0$ exactamente en el punto de silla P_1 → el algoritmo se detiene sin encontrar ningún mínimo. No deseable.

El descenso de gradiente solo garantiza alcanzar un punto donde $\nabla f = 0$, sin distinguir su naturaleza.

In [64]:
```
# Verificacion numerica de la Hessiana de g en los puntos criticos

def hessiana_g(xy):
    # H(g) = [[d2g/dx2, d2g/dxdy], [d2g/dydx, d2g/dy2]] = [[2, 3], [3, 6y]]
    x, y = xy[0], xy[1]
    return [[2.0, 3.0],
            [3.0, 6.0 * y]]

def det_2x2(M):
    return M[0][0]*M[1][1] - M[0][1]*M[1][0]

puntos = [([0.0,    0.0],  "P1 = (0, 0)"),
          ([-9/4,   3/2],  "P2 = (-9/4, 3/2)")]

print(f"{'Punto':>18}  {'det(H)':>10}  {'H[0][0]':>10}  Clasificacion")
print("-" * 65)
for pt, nombre in puntos:
    H  = hessiana_g(pt)
    dH = det_2x2(H)
    h00 = H[0][0]
    if dH < 0:
        tipo = "Punto de silla  (det H < 0)"
    elif dH > 0 and h00 > 0:
        tipo = "Minimo local    (det H > 0, H[0][0] > 0)"
    elif dH > 0 and h00 < 0:
        tipo = "Maximo local    (det H > 0, H[0][0] < 0)"
    else:
        tipo = "Indeterminado"
    print(f"{nombre:>18}  {dH:>10.4f}  {h00:>10.4f}  {tipo}")

print()
for pt, nombre in puntos:
    H = hessiana_g(pt)
```

```
    print(f"H en {nombre}:")
    for fila in H:
        print(f"  {fila}")
    print(f"  nabla_g = {grad_g(pt)}")
    print()
```

```
             Punto      det(H)     H[0][0]  Clasificacion
-----------------------------------------------------------------
       P1 = (0, 0)     -9.0000      2.0000  Punto de silla  (det H < 0)
  P2 = (-9/4, 3/2)      9.0000      2.0000  Minimo local    (det H > 0, H[0][0] >
0)

H en P1 = (0, 0):
  [2.0, 3.0]
  [3.0, 0.0]
  nabla_g = [0.0, 0.0]

H en P2 = (-9/4, 3/2):
  [2.0, 3.0]
  [3.0, 9.0]
  nabla_g = [0.0, 0.0]
```